Functional Programming

Data Wrangling in R

Functional Programming

“R, at its heart, is a functional programming (FP) language. This means that it provides many tools for the creation and manipulation of functions.” - Hadley Wickham

Don’t need to write for-loops! - check this video.

Iterative work

Allows you to flexibly iterate functions to multiple elements of a data object!

Useful when you want to apply a function to:

- lots of columns in a tibble
- multiple tibbles
- multiple data files

base R **apply** functions

Works really simply for all columns, but not a tibble.

```
sapply(mtcars, FUN = round)
```

```
##       mpg cyl disp  hp drat wt qsec vs am gear carb
##  [1,]  21   6  160 110    4  3   16  0  1    4    4
##  [2,]  21   6  160 110    4  3   17  0  1    4    4
##  [3,]  23   4  108  93    4  2   19  1  1    4    1
##  [4,]  21   6  258 110    3  3   19  1  0    3    1
##  [5,]  19   8  360 175    3  3   17  0  0    3    2
##  [6,]  18   6  225 105    3  3   20  1  0    3    1
##  [7,]  14   8  360 245    3  4   16  0  0    3    4
##  [8,]  24   4  147  62    4  3   20  1  0    4    2
##  [9,]  23   4  141  95    4  3   23  1  0    4    2
## [10,]  19   6  168 123    4  3   18  1  0    4    4
## [11,]  18   6  168 123    4  3   19  1  0    4    4
## [12,]  16   8  276 180    3  4   17  0  0    3    3
## [13,]  17   8  276 180    3  4   18  0  0    3    3
## [14,]  15   8  276 180    3  4   18  0  0    3    3
## [15,]  10   8  472 205    3  5   18  0  0    3    4
## [16,]  10   8  460 215    3  5   18  0  0    3    4
## [17,]  15   8  440 230    3  5   17  0  0    3    4
## [18,]  32   4   79  66    4  2   19  1  1    4    1
## [19,]  30   4   76  52    5  2   19  1  1    4    2
## [20,]  34   4   71  65    4  2   20  1  1    4    1
## [21,]  22   4  120  97    4  2   20  1  0    3    1
## [22,]  16   8  318 150    3  4   17  0  0    3    2
## [23,]  15   8  304 150    3  3   17  0  0    3    2
```

Working **across** multiple columns

`across` allows us to perform functions on specific columns more easily. Use with `mutate` or `summarize`.

```
mtcars |>
  mutate(across(
    c(mpg, disp, hp, drat),
    round # no parentheses if no arguments
  ))
```

```
##                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4          21   6  160 110    4 2.620 16.46  0  1    4    4
## Mazda RX4 Wag      21   6  160 110    4 2.875 17.02  0  1    4    4
## Datsun 710         23   4  108  93    4 2.320 18.61  1  1    4    1
## Hornet 4 Drive     21   6  258 110    3 3.215 19.44  1  0    3    1
## Hornet Sportabout  19   8  360 175    3 3.440 17.02  0  0    3    2
## Valiant            18   6  225 105    3 3.460 20.22  1  0    3    1
## Duster 360         14   8  360 245    3 3.570 15.84  0  0    3    4
## Merc 240D          24   4  147  62    4 3.190 20.00  1  0    4    2
## Merc 230           23   4  141  95    4 3.150 22.90  1  0    4    2
## Merc 280           19   6  168 123    4 3.440 18.30  1  0    4    4
## Merc 280C          18   6  168 123    4 3.440 18.90  1  0    4    4
## Merc 450SE         16   8  276 180    3 4.070 17.40  0  0    3    3
## Merc 450SL         17   8  276 180    3 3.730 17.60  0  0    3    3
## Merc 450SLC        15   8  276 180    3 3.780 18.00  0  0    3    3
## Cadillac Fleetwood 10   8  472 205    3 5.250 17.98  0  0    3    4
## Lincoln Continental 10  8  460 215    3 5.424 17.82  0  0    3    4
## Chrysler Imperial  15   8  440 230    3 5.345 17.42  0  0    3    4
## Fiat 128           32   4   79  66    4 2.200 19.47  1  1    4    1
```

Using **across** with arguments

If you want to use arguments, then you need to use the `~` and `.` (or `.x`) as a place holder for what you will be passing into the function.

See documentation for purrr-shortcuts and a comparison of `.` and `.x`.

```
mtcars |>
  mutate(across(
    c(mpg, disp, hp, drat),
    ~ round(., digits = -1) # note ~ and .
  ))
```

```
##                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4          20   6  160 110    0 2.620 16.46  0  1    4    4
## Mazda RX4 Wag      20   6  160 110    0 2.875 17.02  0  1    4    4
## Datsun 710         20   4  110  90    0 2.320 18.61  1  1    4    1
## Hornet 4 Drive     20   6  260 110    0 3.215 19.44  1  0    3    1
## Hornet Sportabout  20   8  360 180    0 3.440 17.02  0  0    3    2
## Valiant            20   6  220 100    0 3.460 20.22  1  0    3    1
## Duster 360         10   8  360 240    0 3.570 15.84  0  0    3    4
## Merc 240D          20   4  150  60    0 3.190 20.00  1  0    4    2
## Merc 230           20   4  140 100    0 3.150 22.90  1  0    4    2
## Merc 280           20   6  170 120    0 3.440 18.30  1  0    4    4
## Merc 280C          20   6  170 120    0 3.440 18.90  1  0    4    4
## Merc 450SE         20   8  280 180    0 4.070 17.40  0  0    3    3
## Merc 450SL         20   8  280 180    0 3.730 17.60  0  0    3    3
## Merc 450SLC        20   8  280 180    0 3.780 18.00  0  0    3    3
## Cadillac Fleetwood 10   8  470 200    0 5.250 17.98  0  0    3    4
## Lincoln Continental 10  8  460 220    0 5.424 17.82  0  0    3    4
```

Writing functions

Why write your own functions?

- Cut down on repetitive code (easier to fix things!)
- Organize code into manageable chunks
- Avoid running code unintentionally

Functions in R

To create a function we can use the `function` function and specify what input the function will take and what it will do to it.

```
my_function <- function(x){x + 1}
my_function
```

```
## function (x)
## {
##     x + 1
## }
## <environment: 0x11ccc3628>
```

```
my_data  <- c(2,3,4)

my_function(x = my_data)
```

```
## [1] 3 4 5
```

Shortcut for functions

Alternatively we can use `\(x)`. See this link about function shortcuts.

```
my_function <- \(x){x + 1}
my_function
```

```
## function (x)
## {
##     x + 1
## }
## <environment: 0x11ccc3628>
```

```
my_function(x = my_data)
```

```
## [1] 3 4 5
```

`purrr` is a super helpful package for iteration!

"Designed to make your functions purrr."

`dplyr` is designed for data frames `purrr` is designed for vectors

The `purrr` package can be very helpful!

- https://purrr.tidyverse.org/
- https://github.com/rstudio/cheatsheets/raw/master/purrr.pdf
- https://jennybc.github.io/purrr-tutorial/

`purrr` functions: `map` and `modify`

applies function to each element of an vector or object

- `map` will output a list
- `map_dbl` will output a vector
- `modify` will output the same object type

purrr functions: map and modify

Apply `sum` to every column

```
mtcars |> map(sum)
```

```
## $mpg
## [1] 642.9
##
## $cyl
## [1] 198
##
## $disp
## [1] 7383.1
##
## $hp
## [1] 4694
##
## $drat
## [1] 115.09
##
## $wt
## [1] 102.952
##
## $qsec
## [1] 571.16
##
## $vs
## [1] 14
##
```

purrr functions: map and modify

Apply `sum` to every column - nicer output

```
mtcars |> map_dbl(sum)
```

```
##      mpg      cyl     disp       hp     drat       wt     qsec       vs
##  642.900  198.000 7383.100 4694.000  115.090  102.952  571.160   14.000
##       am     gear     carb
##   13.000  118.000   90.000
```

purrr functions: map and modify

Apply `round` to every column - get a data.frame back out

```
mtcars |> modify(round)
```

```
##    mpg cyl disp  hp drat wt qsec vs am gear carb
## 1   21   6  160 110    4  3   16  0  1    4    4
## 2   21   6  160 110    4  3   17  0  1    4    4
## 3   23   4  108  93    4  2   19  1  1    4    1
## 4   21   6  258 110    3  3   19  1  0    3    1
## 5   19   8  360 175    3  3   17  0  0    3    2
## 6   18   6  225 105    3  3   20  1  0    3    1
## 7   14   8  360 245    3  4   16  0  0    3    4
## 8   24   4  147  62    4  3   20  1  0    4    2
## 9   23   4  141  95    4  3   23  1  0    4    2
## 10  19   6  168 123    4  3   18  1  0    4    4
## 11  18   6  168 123    4  3   19  1  0    4    4
## 12  16   8  276 180    3  4   17  0  0    3    3
## 13  17   8  276 180    3  4   18  0  0    3    3
## 14  15   8  276 180    3  4   18  0  0    3    3
## 15  10   8  472 205    3  5   18  0  0    3    4
## 16  10   8  460 215    3  5   18  0  0    3    4
## 17  15   8  440 230    3  5   17  0  0    3    4
## 18  32   4   79  66    4  2   19  1  1    4    1
## 19  30   4   76  52    5  2   19  1  1    4    2
## 20  34   4   71  65    4  2   20  1  1    4    1
## 21  22   4  120  97    4  2   20  1  0    3    1
## 22  16   8  318 150    3  4   17  0  0    3    2
## 23  15   8  304 150    3  3   17  0  0    3    2
```

Anonymous functions - "functions on the fly"

You can supply a custom function without naming it. Useful if you only need to use it once!

```
mtcars |> modify(\(x) x * 100)
```

```
##     mpg cyl  disp    hp drat    wt qsec  vs  am gear carb
## 1  2100 600 16000 11000  390 262.0 1646   0 100  400  400
## 2  2100 600 16000 11000  390 287.5 1702   0 100  400  400
## 3  2280 400 10800  9300  385 232.0 1861 100 100  400  100
## 4  2140 600 25800 11000  308 321.5 1944 100   0  300  100
## 5  1870 800 36000 17500  315 344.0 1702   0   0  300  200
## 6  1810 600 22500 10500  276 346.0 2022 100   0  300  100
## 7  1430 800 36000 24500  321 357.0 1584   0   0  300  400
## 8  2440 400 14670  6200  369 319.0 2000 100   0  400  200
## 9  2280 400 14080  9500  392 315.0 2290 100   0  400  200
## 10 1920 600 16760 12300  392 344.0 1830 100   0  400  400
## 11 1780 600 16760 12300  392 344.0 1890 100   0  400  400
## 12 1640 800 27580 18000  307 407.0 1740   0   0  300  300
## 13 1730 800 27580 18000  307 373.0 1760   0   0  300  300
## 14 1520 800 27580 18000  307 378.0 1800   0   0  300  300
## 15 1040 800 47200 20500  293 525.0 1798   0   0  300  400
## 16 1040 800 46000 21500  300 542.4 1782   0   0  300  400
## 17 1470 800 44000 23000  323 534.5 1742   0   0  300  400
## 18 3240 400  7870  6600  408 220.0 1947 100 100  400  100
## 19 3040 400  7570  5200  493 161.5 1852 100 100  400  200
## 20 3390 400  7110  6500  422 183.5 1990 100 100  400  100
## 21 2150 400 12010  9700  370 246.5 2001 100   0  300  100
## 22 1550 800 31800 15000  276 352.0 1687   0   0  300  200
```

modify_if

Using `modify_if()`, we can specify what columns to modify

```
ufo1 <- read_csv("https://sisbid.github.io/Data-Wrangling/data/ufo/ufo_slice_1.csv")
```

```
ufo1 |>
  modify_if(is.character, toupper) |>
  head(3)
```

```
## # A tibble: 3 × 11
##   datetime city  state country shape `duration (seconds)` `duration (hours/min)`
##   <chr>    <chr> <chr> <chr>   <chr>                <dbl> <chr>
## 1 7/28/20… NEW … NY    US      CHAN…                  120 2 MINS
## 2 8/15/19… MEND… NJ    US      FIRE…                   30 30 SECONDS
## 3 10/16/2… CHAR… MI    <NA>    CIGAR                   30 30 SECONDS
## # ℹ 4 more variables: comments <chr>, `date posted` <chr>, latitude <dbl>,
## #   longitude <dbl>
```

Speed test! modify_if vs across

```
system.time(ufo1 |>
              modify_if(is.character, toupper))
```

```
##    user  system elapsed
##   0.001   0.000   0.000
```

```
system.time(ufo1 |>
               mutate(across(where(is.character), toupper)))
```

```
##    user  system elapsed
##   0.002   0.000   0.001
```

Iterative filtering with `if_all`

Previously we filtered for patterns or conditions..

Dilemma: Seems a bit repetitive!

```
mtcars |>
  filter(cyl > 3 & cyl < 8,
         gear > 3 & gear < 8,
         carb > 3 & carb < 8)
```

```
##                mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4     21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag 21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
## Merc 280      19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
## Merc 280C     17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
## Ferrari Dino  19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
```

Now we can filter multiple columns!

`if_all()`: helps us filter on multiple similar conditions simultaneously!

```
mtcars |>
   filter(if_all(c(cyl, gear, carb), ~.x > 3 & .x < 8))
```

```
##                mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4     21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag 21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
## Merc 280      19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
## Merc 280C     17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
## Ferrari Dino  19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
```

Lists

What is a list?

- Lists are the most flexible/“generic” data class in R
- Can be created using list()
- Can hold vectors, strings, matrices, models, list of other lists, lists upon lists!

```
mylist <- list(
  letters = c("A", "b", "c"),
  numbers = 1:3,
  matrix(1:25, ncol = 5),
  iris
)
```

List Structure

```
head(mylist)
```

```
## $letters
## [1] "A" "b" "c"
##
## $numbers
## [1] 1 2 3
##
## [[3]]
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    6   11   16   21
## [2,]    2    7   12   17   22
## [3,]    3    8   13   18   23
## [4,]    4    9   14   19   24
## [5,]    5   10   15   20   25
##
## [[4]]
##     Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
## 1            5.1         3.5          1.4         0.2     setosa
## 2            4.9         3.0          1.4         0.2     setosa
## 3            4.7         3.2          1.3         0.2     setosa
## 4            4.6         3.1          1.5         0.2     setosa
## 5            5.0         3.6          1.4         0.2     setosa
## 6            5.4         3.9          1.7         0.4     setosa
## 7            4.6         3.4          1.4         0.3     setosa
## 8            5.0         3.4          1.5         0.2     setosa
## 9            4.4         2.9          1.4         0.2     setosa
## 10           4.9         3.1          1.5         0.1     setosa
## 11           5.4         3.7          1.5         0.2     setosa
```

List referencing

```
mylist[1] # returns a list
```

```
## $letters
## [1] "A" "b" "c"
```

```
mylist["letters"] # returns a list
```

```
## $letters
## [1] "A" "b" "c"
```

List referencing

```
mylist[[1]] # returns the vector 'letters'
```

```
## [1] "A" "b" "c"
```

```
mylist[["letters"]] # returns the vector 'letters'
```

```
## [1] "A" "b" "c"
```

Why do this at all?

https://jennybc.github.io/purrr-tutorial/bk01_base-functions.html

You need a way to iterate in R in a data-structure-informed way. What does that mean?

You can **iterate over elements of a list**! This might be over sub data frames or different files.

Cleaning up multiple datasets

Store datasets in a list!

```
ufo1 <- read_csv("https://sisbid.github.io/Data-Wrangling/data/ufo/ufo_slice_1.csv")
ufo2 <- read_delim("https://sisbid.github.io/Data-Wrangling/data/ufo/ufo_slice_2.tsv")
ufo3 <- read_delim("https://sisbid.github.io/Data-Wrangling/data/ufo/ufo_slice_3.csv", delim = ":")
```

```
ufo_datasets <- list(ufo1, ufo2, ufo3)
```

Cleaning up multiple datasets

Clean names on all datasets at once:

```
library(janitor)

ufo_datasets_clean <-
  ufo_datasets |>
  map(clean_names)
```

Cleaning up multiple datasets

Confirm columns have been cleaned! Look at `ufo_datasets_clean` or look at each with indexing, eg.:

```
ufo_datasets_clean[[1]]
```

```
## # A tibble: 20 × 11
##    datetime        city  state country shape duration_seconds duration_hours_min
##    <chr>           <chr> <chr> <chr>   <chr>            <dbl> <chr>
##  1 7/28/2002 20:10 new … ny    us      chan…              120 2 mins
##  2 8/15/1988 21:30 mend… nj    us      fire…               30 30 seconds
##  3 10/16/2010 22:… char… mi    <NA>    cigar               30 30 seconds
##  4 4/25/2013 05:30 fran… wi    us      sphe…              360 6 minutes
##  5 1/25/2012 18:22 colo… co    us      cigar              180 3 min.
##  6 3/10/2010 00:23 huson mt    us      circ…              120 several minutes
##  7 8/16/2003 17:00 hous… tx    us      form…             2700 45 minutes
##  8 4/29/2011 20:45 san … ca    us      fire…              600 5-10 mins
##  9 6/30/2004 13:00 char… sc    us      rect…             3600 less than 1 hour
## 10 3/1/2009 14:23  huds… ny    us      sphe…               15 15 seconds
## 11 3/13/2000 20:00 san … ca    us      light               30 15-30 seconds
## 12 9/12/2006 15:30 moja… nv    <NA>    sphe…             1800 30min or more
## 13 9/29/2012 18:30 san … <NA>  <NA>    fire…              600 7-10 minutes
## 14 8/12/2006 00:30 murr… ky    us      chan…            14400 4hr
## 15 9/1/2011 01:55  vash… wa    us      light              900 15 minues
## 16 1/3/2013 00:43  phoe… az    us      <NA>               300 5 minutes
## 17 3/3/2000 04:00  long… fl    us      light               20 20 seconds
## 18 11/29/2013 07:… i-95… sc    <NA>    other               20 20 seconds
## 19 4/5/2014 22:30  san … tx    us      tria…              900 15 minutes
## 20 10/24/2013 19:… lake… tn    us      light               30 30 seconds
```

Cleaning up multiple datasets

Apply an anonymous function! Creates a count `tibble` for each `country`

```
ufo_datasets |>
  map(\(x) x |> count(country))
```

```
## [[1]]
## # A tibble: 2 × 2
##   country     n
##   <chr>   <int>
## 1 us         16
## 2 <NA>        4
##
## [[2]]
## # A tibble: 2 × 2
##   country     n
##   <chr>   <int>
## 1 us         17
## 2 <NA>        3
##
## [[3]]
## # A tibble: 3 × 2
##   country     n
##   <chr>   <int>
## 1 ca          1
## 2 us         15
## 3 <NA>        4
```

List: `group_split()` a dataset

We can create a list by splitting up a dataframe. We will use `mtcars`.

```
head(mtcars)
```

```
##                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
## Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

List: `group_split()` the dataset by `cyl`

The following creates split of data for each unique cyl value:

```
mtcars_split <- mtcars |> group_by(cyl) |> group_split()
mtcars_keys <- mtcars |> group_by(cyl) |> group_keys() |> pull(cyl)
names(mtcars_split) <- mtcars_keys
glimpse(mtcars_split)
```

```
## list<tibble[,11]> [1:3]
## $ 4: tibble [11 × 11] (S3: tbl_df/tbl/data.frame)
## $ 6: tibble [7 × 11] (S3: tbl_df/tbl/data.frame)
## $ 8: tibble [14 × 11] (S3: tbl_df/tbl/data.frame)
## @ ptype: tibble [0 × 11] (S3: tbl_df/tbl/data.frame)
```

List: model on each

```
mtcars_split |>
  map(~lm(mpg ~ wt, data = .)) |> # apply linear model to each
  map(summary) |>
  map_dbl("r.squared")
```

```
##         4         6         8
## 0.5086326 0.4645102 0.4229655
```

Summary

- `function(x){ }` or `\(x){ }` denotes a function.
- `across` works with `mutate` or `summarize`. First specify *what* to work on, then what to *do*. `~` and `.` will help you use function arguments.
- `map` and `modify` **apply functions**. `map` returns a list, `modify` returns the same object type.
- The purrr package has other useful functional programming features.
- lists can be great for storing iterative work.
- `group_split` and `group_keys` can be handy with `group_by` to create a list of subset tibbles.

https://sisbid.github.io/Data-Wrangling/14_Functional_Programming/lab/functional-program-lab.Rmd